Universidade Federal Rural de Pernambuco
Unidade Acadêmica de Belo Jardim

Algoritmos e Estruturas de Dados

Dividir e conquistar

Prof. Waldemar Ferreira **Neto**
Curso: **Engenharia da Computação**
Disciplina: **Algoritmos e Estruturas de Dados**

À PRIMEIRA VISTA, OS PROBLEMAS PARECEM SER INSOLÚVEIS.
O SEGREDO CONSISTE EM DIVIDIR OS PROBLEMAS EM PARTES MENORES, MAIS FÁCEIS DE RESOLVER. FAZENDO ASSIM, É MOLEZA RESOLVER QUALQUER PROBLEMA.
POR EXEMPLO, EU TENHO QUE LER TODO ESTE CAPÍTULO DO LIVRO DE HISTÓRIA. À PRIMEIRA VISTA, PARECE IMPOSSÍVEL. ASSIM, EU VOU DIVIDIR O PROBLEMA.
VOCÊ VAI SE CONCENTRAR NA LEITURA DA PRIMEIRA PÁGINA?
NÃO. VOU ME PERGUNTAR SE VALE A PENA LER ESSA DROGA.

Napoleão

Divisão e conquista

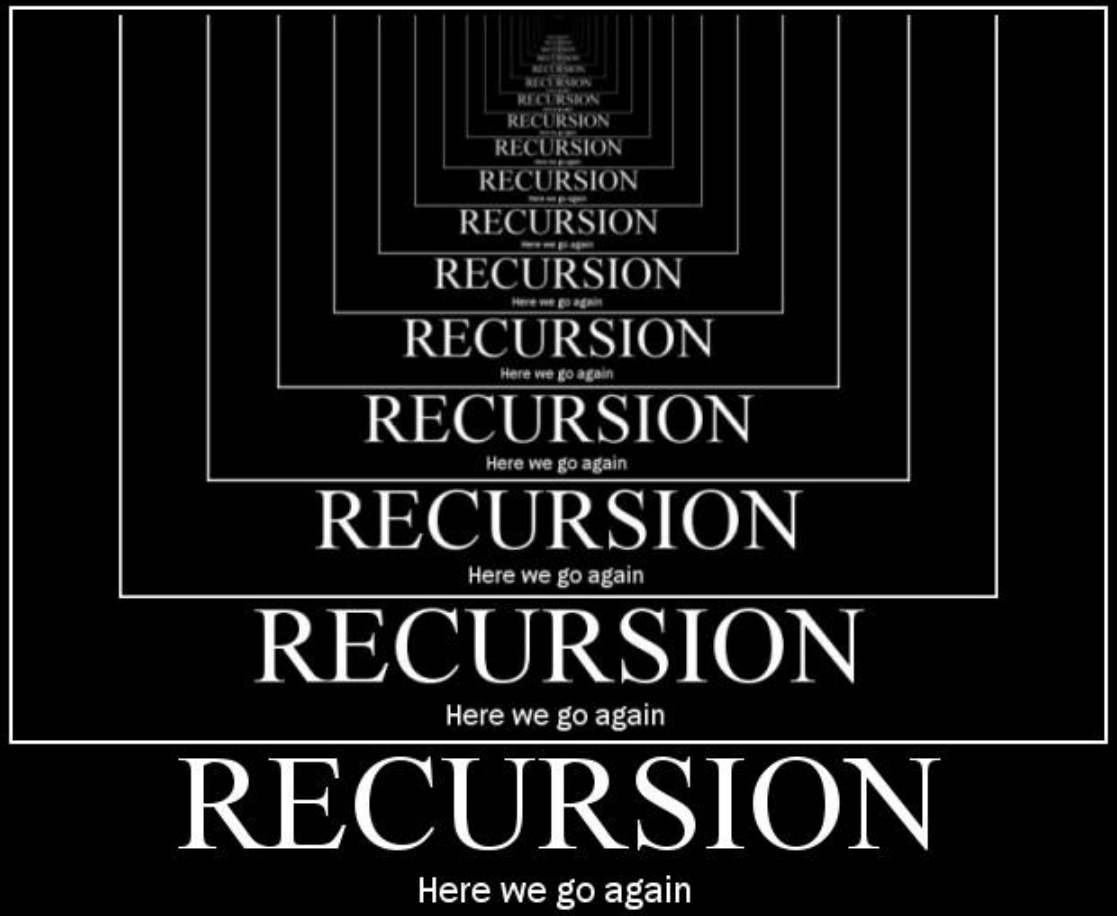
O projeto de muitos algoritmos eficientes é baseado no método da divisão e conquista. Esse método (ou estratégia de projeto de algoritmos) consiste no seguinte:

1. A instância dada do problema é dividida em duas ou mais instâncias menores,
2. Cada instância menor é resolvida usando o próprio algoritmo que está sendo definido,
3. As soluções das instâncias menores são combinadas para produzir uma solução da instância original.

Recursividade

A segunda fase é implementada por uma chamada recursiva. Essa é a fase da conquista.

O método da divisão e conquista produz um algoritmo eficiente se a fase de divisão e a fase da combinação forem suficientemente rápidos.



Problema

Tenho n arquivos digitais no meu computador. Cada arquivo i tem t_i megabytes. Quero gravar o maior número possível de arquivos num pendrive que tem capacidade para c megabytes. O problema pode ser formulado assim: encontrar o maior subconjunto X do intervalo $1..n$ que satisfaça a restrição:

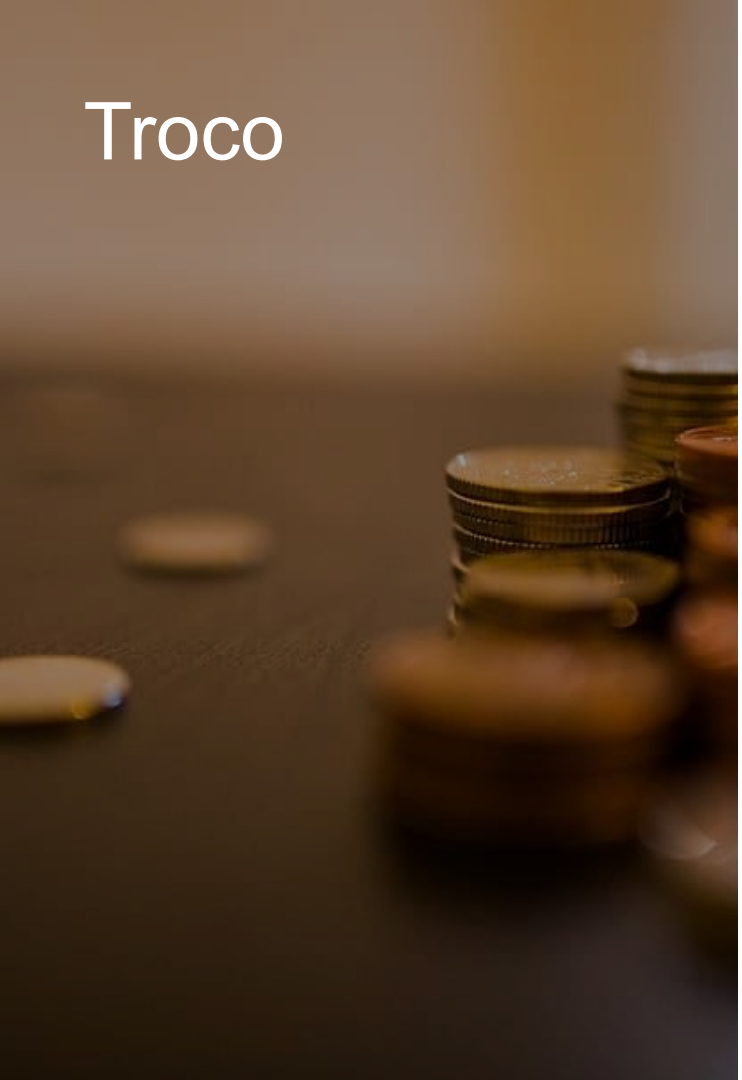
$$\sum_{i \in X} t_i \leq c. \qquad (A)$$

(Note que queremos maximizar $|X|$ e não a soma $\sum_{i \in X} t_i$.)

Exemplo A: A instância do problema do pendrive definida pelos parâmetros n = 8, t = (10, 15, 20, 20, 30, 35, 40, 50) e c = 90 .

A greedy algorithm starts with a solution to a very small subproblem and augments it successively to a solution for the big problem. The augmentation is done in a 'greedy' fashion, that is, paying attention to short-term or local gain, without regard to whether it will lead to a good long-term or global solution. As in real life, greedy algorithms sometimes lead to the best solution, sometimes lead to pretty good solutions, and sometimes lead to lousy solutions. The trick is to determine when to be greedy.

— Ian Parberry, Problems on Algorithms



Troco

O problema do troco (coin change problem) consiste em encontrar a combinação com menor número de moedas cuja soma seja igual a uma quantia determinada, a partir de uma lista de moedas válidas que possuem disponibilidade infinita. Por exemplo, com as denominações padrão do Real brasileiro, há moedas para 1, 5, 10, 25, 50 e 100 centavos: o objetivo é, dada uma quantia (número inteiro, em centavos), encontrar o "caminho" que fornece essa quantia com a menor quantidade de moedas válidas.

Busca Binária

Como encontrar um dado CPF numa longa lista de números de CPF? Se a lista não estiver ordenada, não há o que fazer senão percorrer a lista toda. Se a lista estiver ordenada, entretanto, é possível fazer algo bem melhor. Esse problema é um caso particular do seguinte

Problema da busca em vetor ordenado: Dado um inteiro x e um vetor inteiro crescente $A[1..n]$, encontrar j tal que $A[j-1] < x \leq A[j]$.

O problema faz sentido para todo *n* natural, até mesmo para $n = 0$. Toda instância do problema tem solução e toda solução j pertence ao intervalo $1..n+1$. Se $x > A[n]$, por exemplo, então $n+1$ é a resposta correta. (Veja o exemplo de Busca em vetor ordenado em outra página.)

E a análise Assintótica?

Identificando as primitivas.

- if (n==0 || n == 1) -> avaliação de expressão booleana.
- return 1 -> retorno de método.
- return n -> retorno de método.
- * -> operação aritmética.
- fatorial(n-1) -> ?

```
public int fatorial(int n) {
    if (n==0 || n == 1)
        return 1;
    else
        return n * fatorial(n-1);
}
```

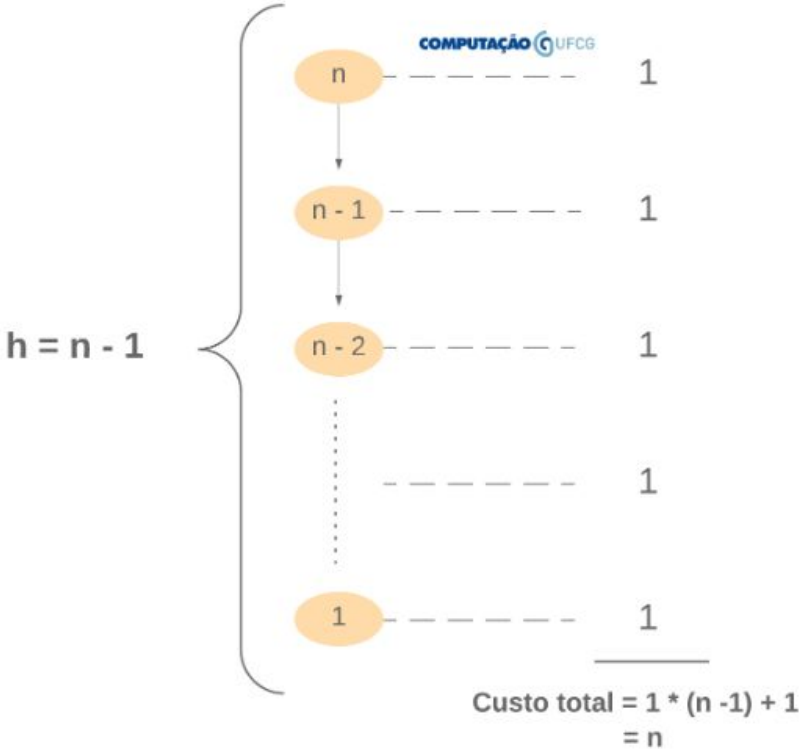
Relação de Recorrência

Relação de recorrência é uma equação ou inequação que descreve uma função em termos dela mesma considerando entradas menores.

Vejamos a relação de recorrência que descreve o algoritmo de cálculo do fatorial:

- $T(n)=T(n-1)+\Theta(1)$,
- simplificando temos: $T(n)=T(n-1)+1$

Árvore Completa



E a análise Assintótica?

$T(n)=T(n/2)+\Theta(1)$.

Simplificando, $T(n)=T(n/2)+1$, onde n = v.length

- $T(n/2)$ representa a chamada recursiva em que a entrada é divida pela metade.
- +1 representa o custo da operação de cálculo do meio e da avaliação das expressões booleanas.

```
public int indexOf(int[] v, int n, int ini, int fim) {
    if (ini < fim) {
        int meio = (ini + fim) / 2;

        if (v[meio] == n) return meio;

        if (n < v[meio])
            return indexOf(v, n, ini, meio - 1);
        else
            return indexOf(v, n, meio + 1, fim);
    } else {
        return -1;
    }
}
```

RESUMO

Não solucionar o problema como um todo, e tentamos solucionar os problema mais simples

Análise Assintótica

1. Faz a análise local
2. Revê quantas vezes pode se repetir (sempre no PIOR caso)

Obrigado!

Any questions?

Contato:

- wpfn@cin.ufpe.br
  (provisório)

Classroom:

- **ulqrpjd**